

Build pipeline model

The objective of this notebook was to build a regression model to predict the price of diamonds based on their physical and visual characteristics. To achieve this goal, we will create and train pipeline as well as testing it. The dataset was loaded, preprocessed, and then we applied pipeline models to predict the price. To ensure the best model is selected, we applied GridSearchCV and observed the error metrics and R2 values.

Importing Libraries

We will start by importing the necessary libraries for building a pipeline model. We will be using the following libraries:

- `pandas` - for data manipulation and analysis
- `numpy` - for numerical operations
- `dill` - for saving the model
- `train_test_split` - for splitting the dataset into training and testing sets
- `Pipeline` - for building a pipeline of transformers and estimators
- `SimpleImputer`, `KNNImputer` - for imputing missing values
- `ColumnTransformer` - for column-wise transformation
- `OneHotEncoder` - for categorical encoding
- `BaseEstimator`, `TransformerMixin` - for creating custom transformers
- `CatBoostRegressor` - for building a CatBoost regression model
- `Pool` - for creating a CatBoost data pool
- `GridSearchCV` - for hyperparameter tuning

```
In [ ]: import pandas as pd
import numpy as np
import dill
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.base import BaseEstimator, TransformerMixin
from catboost import CatBoostRegressor, Pool
from sklearn.model_selection import GridSearchCV
```

Loading Data

Next, we will load the dataset using `pandas read_parquet` function.

```
In [ ]: df = pd.read_csv("../data/filtered_diamonds.csv")
```

Split the dataset into training and testing sets

We will be using the `train_test_split` function from the `sklearn.model_selection` library to split the dataset into a training set and a testing set.

```
In [ ]: X = df.drop("Price", axis=1)
y = df["Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Make pipeline

This is a pipeline that consists of several transformers and an estimator. It is designed to preprocess and model diamond price data. The pipeline has four main stages, which are as follows:

- `imputer`: This transformer replaces missing values in the dataset. It contains two imputers - one for categorical features (using the most frequent value) and another for numeric features (using the KNN algorithm).
- `encoder`: This transformer encodes the categorical features using `OneHotEncoder` for the Shape feature and an `OrdinalConverter` for the remaining categorical features.
- `selector`: This transformer is a custom transformer that uses a `CatBoostRegressor` model to identify the most important features in the dataset. The transformer returns the top 14 features based on their importance.
- `regressor`: This is the final stage of the pipeline and it contains the `CatBoostRegressor` model. The model is used to predict the diamond prices using the selected features.

```
In [ ]: class OrdinalConverter(BaseEstimator, TransformerMixin):
    def __init__(self, order):
        self.order = order
        self.map_func = np.vectorize(lambda x: self.order.index(x))

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        return self.map_func(X)

class CatBoostFeatureSelector(BaseEstimator, TransformerMixin):
    def __init__(self, regressor, num_features):
        self.regressor = regressor
        self.num_features = num_features
        self.feature_importances_ = None

    def fit(self, X, y):
        pool = Pool(X, y, cat_features=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13])
        self.regressor.fit(pool)
        self.feature_importances_ = self.regressor.get_feature_importance(
            pool, type="PredictionValuesChange"
        )
        return self

    def transform(self, X):
        sorted_features = sorted(
            range(len(self.feature_importances_)),
            key=lambda k: -self.feature_importances_[k],
        )
        top_features = sorted_features[: self.num_features]
        return X[:, top_features]

    def get_feature_importances(self, feature_names=None):
        if feature_names is None:
            feature_names = [
                f"feature_{i}" for i in range(len(self.feature_importances_))
            ]
        sorted_importances = sorted(
            zip(feature_names, self.feature_importances_), key=lambda x: -x[1]
        )
        return sorted_importances

imputer = ColumnTransformer(
    transformers=[
```

```

    "cat",
    SimpleImputer(strategy="most_frequent"),
    ["shape", "Clarity", "Colour", "Fluorescence", "Cut", "Polish", "Symmetry"],
),
("num", KNNImputer(n_neighbors=5), ["Weight", "Length", "Width", "Depth"]),
]
)

encoder = ColumnTransformer(
    transformers=[
        ("shape", OneHotEncoder(drop="if_binary", dtype=int), [0]),
        (
            "clarity",
            OrdinalConverter(
                ["I3", "I2", "I1", "SI2", "SI1", "VS2", "VS1", "VS2", "VS1", "IF"]
            ),
            [1],
        ),
        (
            "color",
            OrdinalConverter(
                [
                    "Y-Z",
                    "W-X",
                    "W",
                    "U-V",
                    "S-T",
                    "Q-R",
                    "O-P",
                    "O",
                    "N",
                    "M",
                    "L",
                    "K",
                    "J",
                    "I",
                    "H",
                    "G",
                    "F",
                    "E",
                    "D",
                    "FANCY",
                ]
            ),
            [2],
        ),
        (
            "fluorescence",
            OrdinalConverter(["N", "M", "F", "SL", "ST", "VS", "VSL"]),
            [3],
        ),
        ("cat", OrdinalConverter(["FR", "GD", "VG", "EX"]), [4, 5, 6]),
    ],
    remainder="passthrough",
)

selector = CatBoostFeatureSelector(
    regressor=CatBoostRegressor(verbose=0, allow_writing_files=False), num_features=14
)

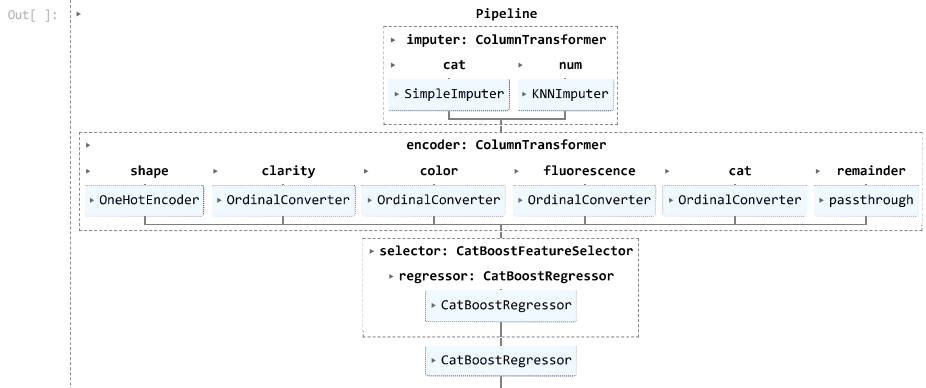
regressor = CatBoostRegressor(verbose=0, allow_writing_files=False)

pipeline = Pipeline(
    steps=[
        ("imputer", imputer),
        ("encoder", encoder),
        ("selector", selector),
        ("regressor", regressor),
    ]
)

```

Train the model

In []: `pipeline.fit(X_train, y_train)`



Testing the model using train dataset

In []: `pipeline.score(X_train, y_train)`

Out []: 0.9905597836052875

Testing the model using test dataset

In []: `pipeline.score(X_test, y_test)`

Out []: 0.8827497503683435

Hyperparameters tuning

We will use GridSearchCV to hyperparameters tuning. Parameters which are tuning are `selector__num_features`, `regressor__learning_rate`, `regressor__depth` and `regressor__l2_leaf_reg`, with `cv=3`. And we will use `r2` for scoring our performance

```
In [ ]: # Define the hyperparameters to tune
```

```
params = {  
    "selector__num_features": [10, 12, 14, 16, 18],  
    "regressor__learning_rate": [0.01, 0.05, 0.1],  
    "regressor__depth": [4, 6, 8],  
    "regressor__l2_leaf_reg": [1, 3, 5],  
}
```

```
In [ ]: # Create the GridSearchCV object
```

```
grid_search = GridSearchCV(  
    pipeline, param_grid=params, cv=3, scoring="r2", n_jobs=-1, verbose=3  
)
```

```
In [ ]: # Fit the GridSearchCV object to the data
```

```
grid_search.fit(X_train, y_train)
```

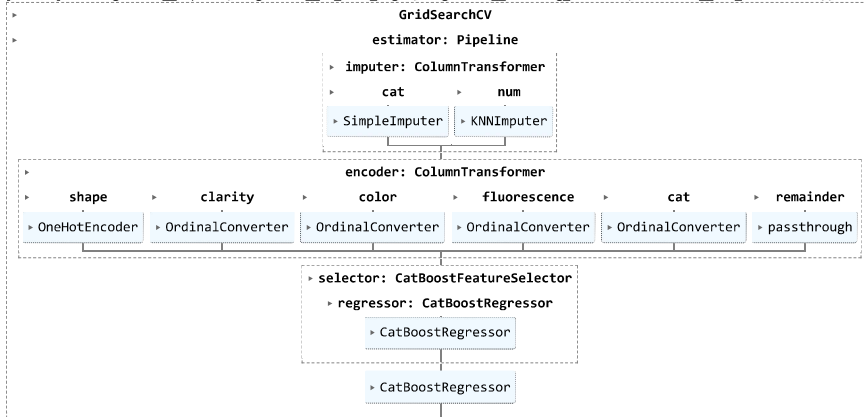
Fitting 3 folds for each of 135 candidates, totalling 405 fits

[illegible]

[illegible]

[illegible]

Out[]:



In []:

```
# Print the best hyperparameters and corresponding score
print(f"Best score: {grid_search.best_score_:.4f}")
print(f"Best parameters: {grid_search.best_params_}")
```

Best score: 0.9029

Best parameters: {'regressor__depth': 8, 'regressor__l2_leaf_reg': 5, 'regressor__learning_rate': 0.05, 'selector__num_features': 14}

By applying this, we improve the score of our model and above parameters are best tuned for our model

Final pipeline 🚀

We will use best parameters and build our final pipeline model

In []:

```

imputer = ColumnTransformer(
    transformers=[
        (
            "cat",
            SimpleImputer(strategy="most_frequent"),
            ["Shape", "Clarity", "Colour", "Fluorescence", "Cut", "Polish", "Symmetry"],
        ),
        ("num", KNNImputer(n_neighbors=5), ["Weight", "Length", "Width", "Depth"]),
    ]
)

encoder = ColumnTransformer(
    transformers=[
        ("shape", OneHotEncoder(drop="if_binary", dtype=int), [0]),
        (
            "clarity",
            OrdinalConverter(
                ["I3", "I2", "I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"],
                [1],
            ),
        ),
        (
            "color",
            OrdinalConverter(
                [
                    "Y-Z",
                    "W-X",
                    "W",
                    "U-V",
                    "S-T",
                    "Q-R",
                    "O-P",
                    "O",
                    "N",
                    "M",
                    "L",
                    "K",
                    "J",
                    "I",
                    "H",
                    "G",
                    "F",
                    "E",
                    "D",
                    "FANCY",
                ],
            ),
        ),
    ],
    [2],
)

```

```

        "fluorescence",
        OrdinalConverter(["N", "M", "F", "SL", "ST", "VS", "VSL"]),
        [3],
    ),
    ("cat", OrdinalConverter(["FR", "GD", "VG", "EX"]), [4, 5, 6]),
],
remainder="passthrough",
)

selector = CatBoostFeatureSelector(
    regressor=CatBoostRegressor(verbose=0, allow_writing_files=False), num_features=14
)

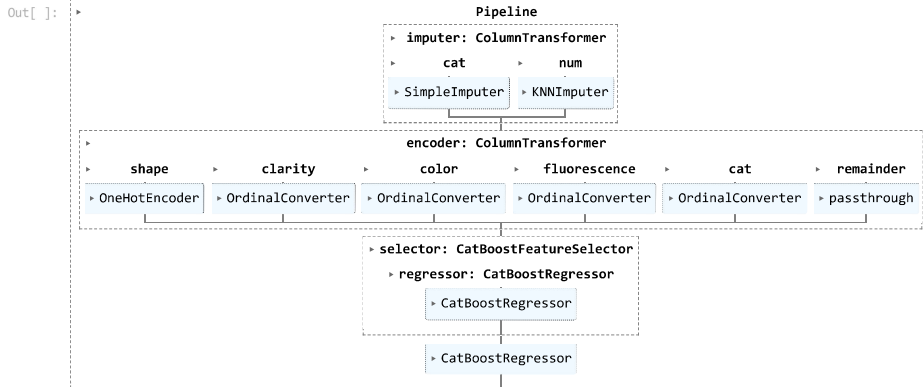
regressor = CatBoostRegressor(
    verbose=0, allow_writing_files=False, depth=8, l2_leaf_reg=5, learning_rate=0.05
)

pipeline = Pipeline(
    steps=[
        ("imputer", imputer),
        ("encoder", encoder),
        ("selector", selector),
        ("regressor", regressor),
    ]
)

```

Train our model

```
In [ ]: pipeline.fit(X_train, y_train)
```



Testing the model using train dataset

```
In [ ]: pipeline.score(X_train, y_train)
```

```
Out [ ]: 0.9916820007381009
```

Testing the model using test dataset

```
In [ ]: pipeline.score(X_test, y_test)
```

```
Out [ ]: 0.8802009811349544
```

Saving the model 🗄️

Save the model using dill

```
In [ ]: # Save the trained pipeline to a file
with open("../data/model.br", "wb") as f:
    dill.dump(pipeline, f)
```

Testing our model 🧪

We test the model be the user

Loading model 📁

Next, we will load the model using dill.

```
In [ ]: with open("../data/model.br", "rb") as f:
    model = dill.load(f)
```

Create a static user input

```
In [ ]: # Create a pandas dataframe from the input data
input_data = [
    "PRINCESS",
    0.3100586,
    "VVS1",
    "D",
    "EX",
    "EX",
    "GD",
    "N",
    3.8496094,
    3.7695312,
    2.6894531,
]
columns = [
    "Shape",
    "Weight",
    "Clarity",
    "Colour",
    "Cut",
    "Polish",
    "Symmetry",
    "Fluorescence",
    "Length",
]
```

```
        "Width",
        "Depth",
    ]

    input_df = pd.DataFrame([input_data], columns=columns)
```

Predict the output

```
In [ ]: model.predict(input_df)
```

```
Out[ ]: array([1085.27686875])
```

Create user interface

```
In [ ]: # Define the input form
print("Please enter the following details for the diamond:")
shape = str(
    input(
        "Shape ('PRINCESS', 'ROUND', 'HEART', 'OVAL', 'CUSHION', 'EMERALD', 'MARQUISE', 'PEAR'): "
    )
)
weight = float(input("Carat weight (e.g. 1.23): "))
clarity = str(
    input(
        "Clarity ('I3', 'I2', 'I1', 'SI2', 'SI1', 'VS2', 'VS1', 'VVS2', 'VVS1', 'IF'): "
    )
)
color = str(
    input(
        "Color ('Y-Z', 'W-X', 'W', 'U-V', 'S-T', 'Q-R', 'O-P', 'O', 'N', 'M', 'L', 'K', 'J', 'I', 'H', 'G', 'F', 'E', 'D', 'FANCY'): "
    )
)
cut = str(input("Cut ('FR', 'GD', 'VG', 'EX'): "))
polish = str(input("Polish ('FR', 'GD', 'VG', 'EX'): "))
symmetry = str(input("Symmetry ('FR', 'GD', 'VG', 'EX'): "))
fluorescence = str(input("Fluorescence ('N', 'M', 'F', 'SL', 'ST', 'VS', 'VSL'): "))
x = float(input("Length in mm (e.g. 6.55): "))
y = float(input("Width in mm (e.g. 6.51): "))
z = float(input("Depth in mm (e.g. 4.11): "))
```

Please enter the following details for the diamond:

```
In [ ]: input_data = [
    shape,
    weight,
    clarity,
    color,
    cut,
    polish,
    symmetry,
    fluorescence,
    x,
    y,
    z,
]

columns = [
    "Shape",
    "Weight",
    "Clarity",
    "Colour",
    "Cut",
    "Polish",
    "Symmetry",
    "Fluorescence",
    "Length",
    "Width",
    "Depth",
]
```

```
In [ ]: # Create a pandas dataframe from the input data
input_df = pd.DataFrame([input_data], columns=columns)
```

```
In [ ]: output = model.predict(input_df)
```

```
In [ ]: # Print the predicted output
print("The predicted price of the diamond is $", round(output[0], 2))
```

The predicted price of the diamond is \$ 11272.6

Conclusion 🎉

In this notebook, we build our pipeline model that consists of several transformers and an estimator. It is designed to preprocess and model diamond price data. The pipeline has four main stages, which are as follows:

- `imputer`: This transformer replaces missing values in the dataset. It contains two imputers - one for categorical features (using the most frequent value) and another for numeric features (using the KNN algorithm).
- `encoder`: This transformer encodes the categorical features using OneHotEncoder for the Shape feature and an OrdinalConverter for the remaining categorical features.
- `selector`: This transformer is a custom transformer that uses a CatBoostRegressor model to identify the most important features in the dataset. The transformer returns the top 14 features based on their importance.
- `regressor`: This is the final stage of the pipeline and it contains the CatBoostRegressor model. The model is used to predict the diamond prices using the selected features.

We also applied hyper tuning using GridSearchCV for our model to improve its performances and saved our model in pickle format

Finally, we also implemented user interface to test and demonstrate the model.